

RiskRadar — ML / Data Engineering Progress Update

#4

Author: Ire (ML / Data Engineer) **Covers:** Shared feature-calculation package, model contract for Chidera, and a dataset limitation discovered while validating it **Status:** Fourth addition — calibration and alert-budget decision still pending

1. Overview

This update covers building the piece the PRD requires but did not yet exist: a single, shared feature-calculation package that both training and Chidera's live backend can call, so the two sides can never quietly drift apart (PRD section 9.4). While validating this package against our real data, we found and diagnosed a genuine dataset limitation, and made an explicit decision about which model to treat as authoritative going forward.

2. The shared feature package

File: `ml/features.py`

- Defines `Transaction` (raw transaction fields, matching the PRD's transaction API contract) and `CustomerState` (a customer's rolling history — amounts, timestamps, known devices, last location).
- One function, `compute_features(transaction, state)`, computes all 24 model features from these raw inputs. This is the single implementation both training and live scoring must use — no separate notebook-only version.
- `-1` is used consistently to mean "does not apply" (e.g. no prior transaction yet), rather than silently defaulting to 0.

3. Test suite proving training/live parity

File: `tests/test_features.py` — 8 tests, all passing.

The key test (`test_batch_and_incremental_calls_produce_identical_results`) is the actual proof the PRD asks for: computing features one transaction at a time (the way live scoring would) produces byte-for-byte identical output to computing them in a batch loop (the way training does) — because both call the exact same function. Other tests cover the "-1 means does not apply" convention, new-device detection, impossible-travel detection, and determinism across repeated runs.

4. Model contract for Chidera

File: `docs/model-contract.md`

Written as the actual interface document Chidera needs: the exact `Transaction` / `CustomerState` shapes, the model's output format (`fraud_probability` , `model_version` , `top_contributing_features`), and three limitations his decision policy must design around — no single usable ML threshold exists yet (rules must narrow the alert volume), the "rules" currently live inside feature computation rather than as separate outputs (but are readable directly off the feature dict if needed separately), and the model is not yet calibrated.

5. Regenerating training data through the new module — and what we found

To fully close the training/live parity gap, we attempted to regenerate the training data by running the raw dataset through `compute_features()` directly, instead of using the HuggingFace dataset's own pre-baked aggregate columns.

First attempt: selected ~193,000 customers at random to reach 1,000,000 rows. Result: `is_ato_risk` (previously the single most important feature, 36.5% of the v1 model's decision-making) dropped to **zero importance**.
Diagnosis: median customer history length was only 2 transactions — with so little history, nearly every transaction looks "new" by default, which isn't a real signal, just a cold-start artifact.

Second attempt: restricted selection to customers with at least 8 transactions (177,406 such customers exist, more than enough to reach 1,000,000 rows). Result: `is_ato_risk` **still showed zero importance**.

Root cause found: `device_seen_count` was `0` for every single one of the 1,000,002 rows — meaning `device_hash` never repeated for the same customer across their history, not once. This means `device_hash` in this dataset is not a stable, persistent per-customer device identifier; it behaves as if generated independently per transaction. The original dataset's `new_device_transaction` column (which powered the v1 model's strong result) was evidently computed by the dataset's own generator using internal logic we don't have access to — `device_hash` and `new_device_transaction` are effectively decorrelated fields in this dataset.

6. Decision

This is a **limitation of this specific public dataset**, not a flaw in `m1/features.py`'s design. In a real bank, `device_id` would be a genuine persistent fingerprint that does repeat across a customer's real sessions — `compute_features()`'s logic is correct for that real-world case. It simply cannot be validated against this dataset's synthetic `device_hash` field.

Decision: `models/fraud_model_v1.pkl` (trained in `05_gbm_baseline.ipynb` on the dataset's own pre-baked features, with the four leaky columns removed) remains the **officially reported model**. The shared feature package (`m1/features.py`) is kept as-is and is considered correct for production use with real device data — this is documented as a disclosed gap between what the module will do with real bank data and what can currently be proven using this particular public dataset, rather than either silently ignored or used to justify a worse-performing model.

7. Summary table

Task	Status
Shared feature-calculation package	Done (<code>m1/features.py</code>)
Test suite proving training/live parity	Done, 8/8 passing
Model contract document for Chidera	Done (<code>docs/model-contract.md</code>)
Regenerate training data via shared module	Attempted; dataset limitation found and documented
Decide authoritative model	Done — v1 (dataset features) remains official

8. What's next (fifth addition)

- **Calibration:** wrap `fraud_model_v1.pkl` so its stated probability matches its actual accuracy where alerts are raised.

- **Alert-budget decision:** bring the threshold-sweep evidence to the team so an operating threshold can be chosen (PRD section 8.9).
- Confirm with Chidera whether the live backend will have access to a real, persistent `device_id` — if so, `m1/features.py`'s ATO logic will work correctly in production even though it couldn't be proven on this dataset.